

```

import time, re, json
from dspm import DSPMMemory
from dspm.patch import count_tokens

JUDGE_MODEL = "qwen/qwen3.8-27b"
EXTRACTOR_MODEL = "openai/gpt-oss-120b"
BUDGET = 80 # tight on purpose: the 250 budget doesn't bind on these short conversations

# ----- judge -----
def judge(constraint, context):
    prompt = (
        "Does this constraint appear in the context below, even if paraphrased? "
        "If the constraint was revised, the context must show the NEW value as current.\n\n"
        f"CONSTRAINT: {constraint}\n\nCONTEXT: \n{context}\n\n"
        "Answer with exactly one word: YES or NO"
    )
    try:
        r = extractor_llm.chat.completions.create(
            model=JUDGE_MODEL, temperature=0, max_tokens=600, timeout=40,
            messages=[{"role": "user", "content": prompt}]
        )
        txt = r.choices[0].message.content or ""
        txt = re.sub(r"<think>.*?</think>", "", txt, flags=re.S).strip().upper()
        hits = re.findall(r"\b(YES|NO)\b", txt)
        if not hits:
            return None # unparseable or truncated -> counted as error, not as NO
        return hits[-1] == "YES"
    except Exception as e:
        print(" judge error:", str(e)[:100])
        return None

# ----- baseline: last-K turns that fit the budget -----
def lastk(conv, budget):
    kept, used = [], 0
    for role, text in reversed(conv):
        line = f"{role}: {text}"
        t = count_tokens(line)
        if used + t > budget:
            break
        kept.insert(0, line); used += t
    return "\n".join(kept)

# ----- sanity check: judge must say NO to something never stated -----
fake = "The system must use MongoDB as its database"
real_ctx = results["coding_agent"]["context"]
print("Sanity (expect False):", judge(fake, real_ctx))
print("Sanity (expect True): ", judge("PostgreSQL 16", real_ctx))
print()

# ----- main test -----
summary = {"dspm": [0, 0, 0], "lastk": [0, 0, 0]} # [yes, judged, errors]
detail = {}

for name, conv, gt in ALL_CONVS:
    print(f"{name}: extracting with DSPM (budget {BUDGET})...", flush=True)
    m = DSPMMemory(budget=BUDGET, llm_client=extractor_llm, model=EXTRACTOR_MODEL)
    for role, text in conv:
        m.add_turn(role, text)
    d_ctx = m.get_context(query="What are all the rules, constraints, and current values?")
    l_ctx = lastk(conv, BUDGET)

    detail[name] = {"dspm_ctx": d_ctx, "lastk_ctx": l_ctx, "dspm": [], "lastk": []}
    for label, ctx in (("dspm", d_ctx), ("lastk", l_ctx)):
        for c in gt:
            v = judge(c, ctx)
            detail[name][label].append(v)
            if v is None: summary[label][2] += 1
            else:
                summary[label][1] += 1
                summary[label][0] += int(v)
        dv, lv = detail[name]["dspm"], detail[name]["lastk"]
        print(f" DSPM {sum(v is True for v in dv)}/{len(gt)} ({count_tokens(d_ctx)} tok)")
        print(f" last-K {sum(v is True for v in lv)}/{len(gt)} ({count_tokens(l_ctx)} tok)", flush=True)

print("\n" + "=" * 50)
print(f"Extractor: {EXTRACTOR_MODEL} | Judge: {JUDGE_MODEL} | Budget: {BUDGET}")

```

```
for label in ("dspm", "lastk"):
    y, n, e = summary[label]
    print(f"{label}>6: {y}/{n} retained (judge errors: {e})")
print("=" * 50)

json.dump({"budget": BUDGET, "extractor": EXTRACTOR_MODEL, "judge": JUDGE_MODEL,
          "summary": summary, "detail": detail},
          open("budget80_results.json", "w"), indent=2)
print("Saved budget80_results.json")
```

Sanity (expect False): False
Sanity (expect True): True

coding_agent: extracting with DSPM (budget 80)...

Loading weights: 100% 103/103 [00:00<00:00, 3212.21it/s]

DSPM 7/7 (74 tok)

last-K 3/7 (70 tok)

support_bot: extracting with DSPM (budget 80)...

Loading weights: 100% 103/103 [00:00<00:00, 3403.13it/s]

DSPM 5/7 (79 tok)

last-K 3/7 (67 tok)

fintech: extracting with DSPM (budget 80)...

Loading weights: 100% 103/103 [00:00<00:00, 4178.89it/s]

DSPM 5/5 (80 tok)

last-K 3/5 (71 tok)

=====
Extractor: openai/gpt-oss-120b | Judge: qwen/qwen3.8-27b | Budget: 80
dspm: 17/19 retained (judge errors: 0)
lastk: 9/19 retained (judge errors: 0)
=====

Saved budget80_results.json